

NATIONAL UNIVERSITY

551 M.F. JHOCSO ST., SAMPALOC, MANILA

COLLEGE OF ENGINEERING

Name/Section	Lance C. Ortiz / COE 232	Rating
Date performed	August 13, 2026	
Date submitted	August 20, 2026	

SOFTWARE DESIGN EXPERIMENT NO. 4

Advanced Desktop Authentication, Account Security, and Input Validation

I. OBJECTIVES:

By the end of this activity, students should be able to:

1. Implement secure desktop authentication and user account controls to prevent unauthorized access.
2. Apply appropriate account security hardening techniques to strengthen desktop system protection.
3. Implement and evaluate robust input validation techniques to ensure secure and reliable user input handling.

II. COURSE LEARNING OUTCOMES (CLOs)

1. Apply fundamental cybersecurity principles in implementing secure authentication, account management, and access control mechanisms.
2. Configure and implement appropriate security measures to protect desktop systems against unauthorized access and common security threats.
3. Evaluate and apply secure input validation techniques to improve the reliability, integrity, and security of software applications.

III. SOFTWARE AND TOOLS REQUIRED

Visual Studio Code (VS Code)

SQLite DB

A terminal or command prompt

SQLite

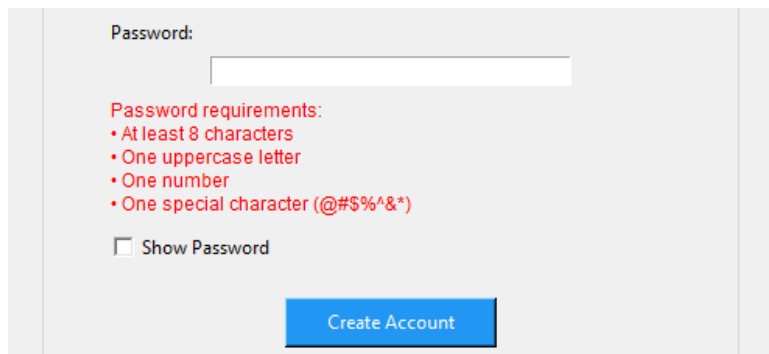
IV. PROCEDURE & SOURCE CODE

A. Experimental Tasks

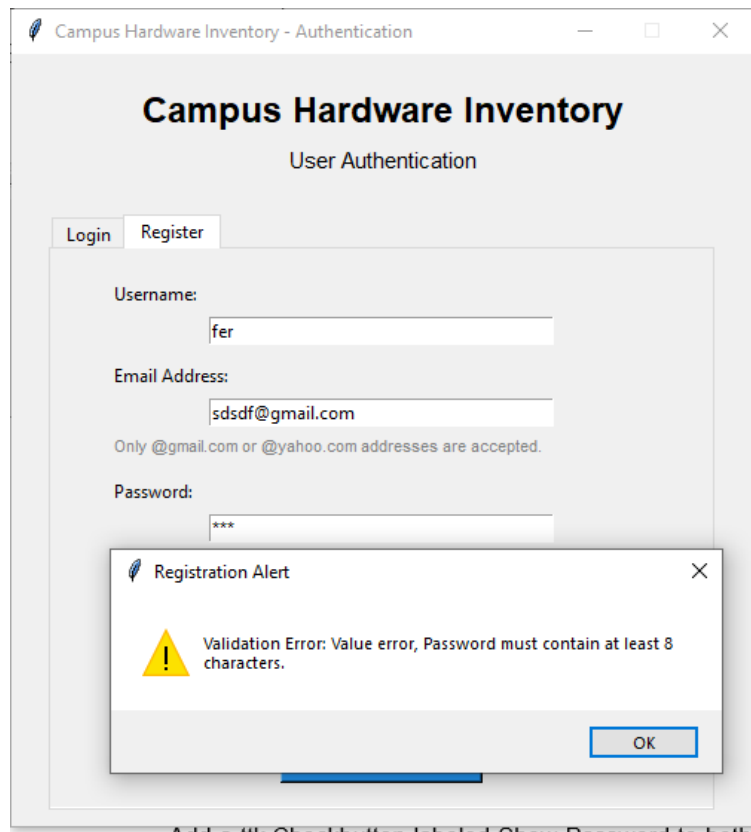
The following security enhancements shall be implemented in the existing authentication system and subsequently applied to the Campus Hardware Inventory System supplementary project.

Task 1: Implement Password Complexity Validation

- Update models/schemas.py.
- Implement Pydantic validation with a regular expression requiring:
 - At least 8 characters.
 - At least one uppercase letter ([A-Z]).
 - At least one number ([0-9]).
 - At least one special character from [@\$%^&*].
- Test the registration form using both valid and invalid passwords.
- Verify that passwords failing any requirement are rejected before being saved.

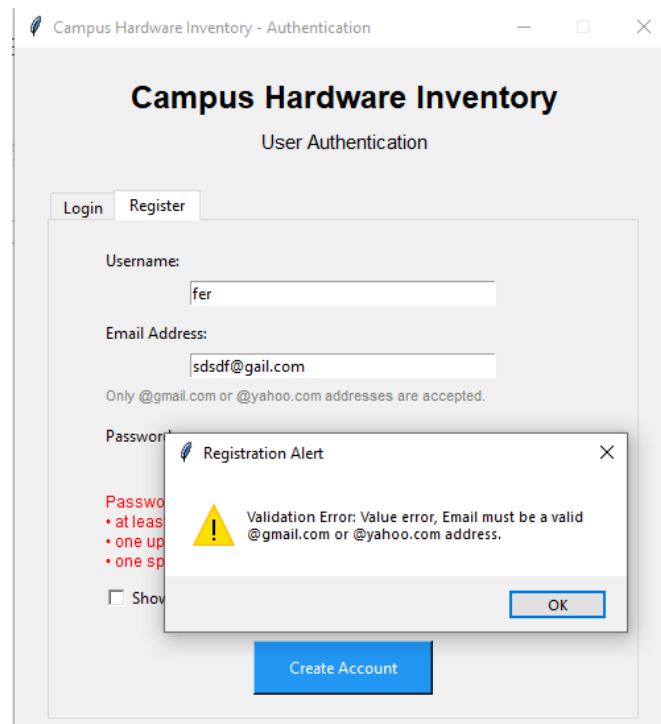


The screenshot shows a web form for creating an account. It features a 'Password:' label above a text input field. Below the input field, the text 'Password requirements:' is followed by a bulleted list of requirements: 'At least 8 characters', 'One uppercase letter', 'One number', and 'One special character (@#\$%^&*)'. There is a checkbox labeled 'Show Password' and a blue 'Create Account' button at the bottom.



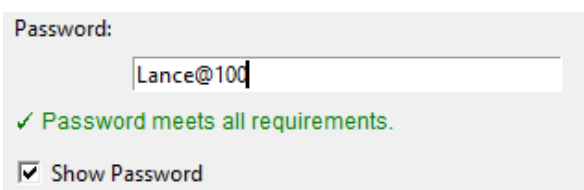
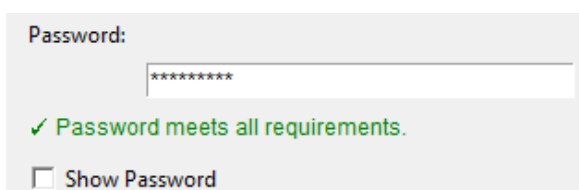
Task 2: Integrate Email Address Validation

- Modify the SQLite users table to include: email TEXT UNIQUE NOT NULL.
- Add an Email Address field to the registration form in views/login_view.py.
- Implement email format validation using Pydantic and/or regular expressions.
- Test valid and invalid email formats.
- Verify that duplicate email addresses cannot be registered.



Task 3: Implement Show/Hide Password Functionality

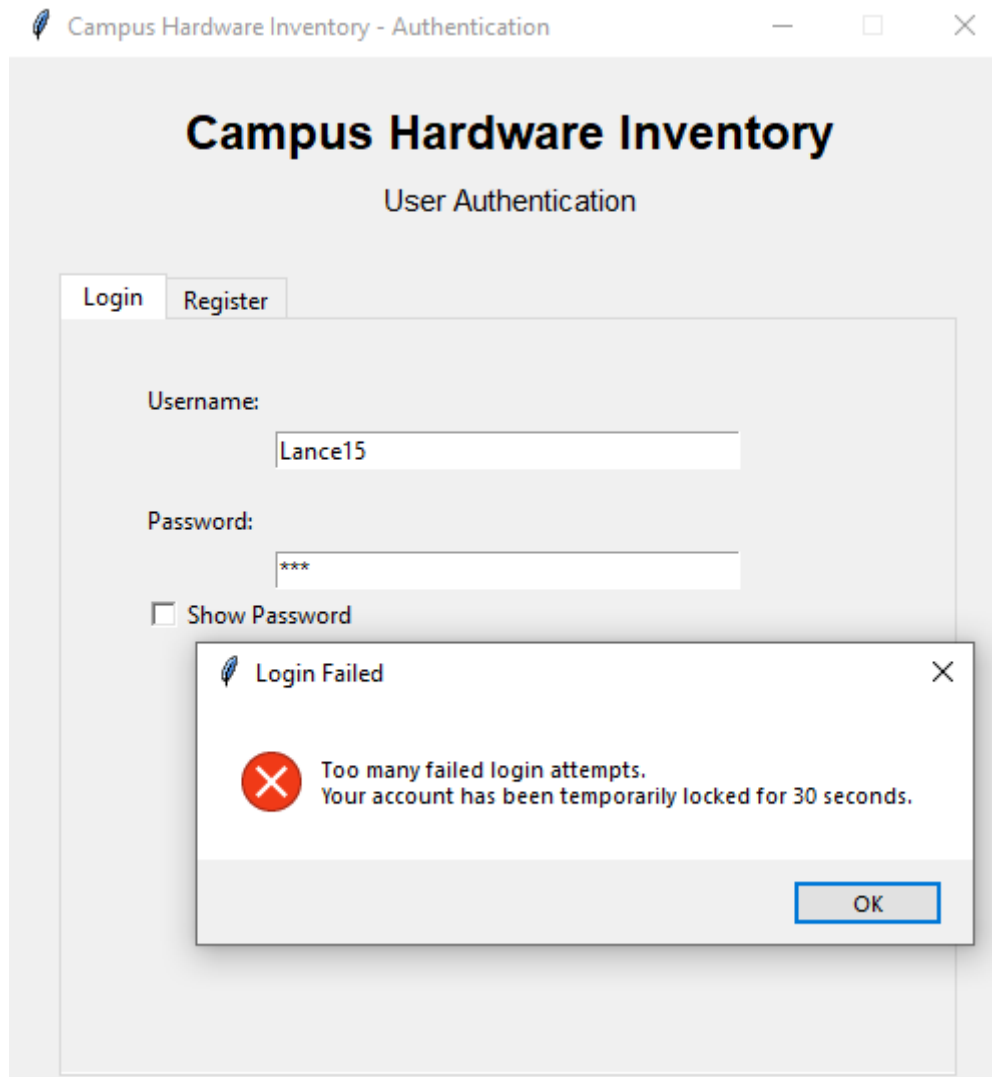
- Add a ttk.Checkbutton labeled Show Password to both the Login and Register forms.
- Configure the password field to display * when hidden.
- Implement dynamic switching between masked (*) and plain-text (""") display.
- Verify that the toggle works correctly on both authentication screens.



Task 4: Implement Anti-Brute-Force Account Lockout

- Modify controllers/auth_controller.py.
- Track consecutive failed login attempts for each username.
- Lock the account after 3 consecutive failed login attempts.
- Set the temporary lockout duration to exactly 30 seconds.

- Display an appropriate warning and countdown timer during the lockout period.
- Prevent further login attempts while the account is locked.
- Record lockout events and timestamps in app_logging/app.log.
- Verify that the account becomes accessible again after the 30-second lockout period.



Task 5: Apply Security Enhancements to the Supplementary Project

Apply the same four security requirements to the Campus Hardware Inventory System:

1. Password Complexity Validation
2. Email Address Integration and Validation

3. Show/Hide Password Toggle

4. 3-Failed-Attempts / 30-Second Account Lockout

Students must demonstrate that each security feature is functional and properly integrated into the system.

Campus Hardware Inventory
User Authentication

Login Register

Username:

Password:

☐ Show Password

Login

Username:

Email Address:

Only @gmail.com or @yahoo.com addresses are accepted.

Password:

Password requirements:

- At least 8 characters
- One uppercase letter
- One number
- One special character (@#\$\$%^&*)

☒ Show Password

Create Account

Campus Hardware Inventory - User: Lance15

Total Asset Value: \$22,220.00

Add New Equipment

Name: Category:

Quantity: Unit Price (\$):

Save Item

Status Legend

- In Stock (> 5)
- Low Stock (1-5)
- Out of Stock (0)

ID	Name	Category	Qty	Price	Status
1	ESP 32	Microcontroller	0	300.0	Out of Stock
2	LED	MISC	0	5.0	Out of Stock
3	Jumper	Wires	10	5.0	In Stock
4	GTX 1080	GPU	2	2000.0	Low Stock
5	Keyboard	5	5	400.0	Low Stock
6	q	w	1	100.0	Low Stock
7	k	1	12	2.0	In Stock
9	dsad	hg	5	5.0	Low Stock
11	Holaa	ds	10	22.0	In Stock
14	Arduino	Microcontroller	10	1000.0	In Stock
15	cc	misc	1	1.0	Low Stock
16	Buzzer	Misc	10	20.0	In Stock
17	Mems Microphone	Input Device	2	300.0	Low Stock
18	Mouse	Misc	10	500.0	In Stock

Update Selected Record

New Quantity: New Unit Price (\$):

Update Record

Export Inventory to CSV

Logout

```
import sqlite3
```

```

import os
import csv
import logging
import re
import time
from datetime import datetime
import tkinter as tk
from tkinter import ttk, messagebox

from pydantic import BaseModel, Field, field_validator, ValidationError
import bcrypt

# =====
# 1. CENTRAL LOGGING SETUP
# =====

def setup_logger():
    log_dir = "app_logging"

    if not os.path.exists(log_dir):
        os.makedirs(log_dir)

    logging.basicConfig(
        filename=os.path.join(log_dir, "app.log"),
        level=logging.INFO,
        format="%(asctime)s - [%(levelname)s] - %(name)s - %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S"
    )

    return logging.getLogger("CampusHardwareLogger")

logger = setup_logger()

# =====
# 2. DATABASE INITIALIZATION
# =====

DB_NAME = "hardware_inventory.db"

def init_db(db_name=DB_NAME):
    try:
        conn = sqlite3.connect(db_name)
        cursor = conn.cursor()

        # -----
        # Users Table
        # -----

```

```

cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT,
        password_hash TEXT NOT NULL,
        failed_attempts INTEGER DEFAULT 0,
        locked_until REAL DEFAULT 0
    )
""")

# -----
# Database Migration
# -----
# This allows older databases to work even if
# they were created using the original schema.

cursor.execute("PRAGMA table_info(users)")
existing_columns = [column[1] for column in cursor.fetchall()]

if "email" not in existing_columns:
    cursor.execute("ALTER TABLE users ADD COLUMN email TEXT")
    logger.info("Database migration: Added email column.")

if "failed_attempts" not in existing_columns:
    cursor.execute(
        "ALTER TABLE users ADD COLUMN failed_attempts INTEGER DEFAULT 0"
    )
    logger.info("Database migration: Added failed_attempts column.")

if "locked_until" not in existing_columns:
    cursor.execute(
        "ALTER TABLE users ADD COLUMN locked_until REAL DEFAULT 0"
    )
    logger.info("Database migration: Added locked_until column.")

# -----
# Hardware Inventory Table
# -----

cursor.execute("""
    CREATE TABLE IF NOT EXISTS hardware (
        item_id INTEGER PRIMARY KEY AUTOINCREMENT,
        item_name TEXT UNIQUE NOT NULL,
        category TEXT NOT NULL,
        quantity INTEGER NOT NULL,
        unit_price REAL NOT NULL,
        status TEXT NOT NULL
    )
""")

```

```

        conn.commit()
        conn.close()

    logger.info("Database initialized successfully.")

except sqlite3.Error as e:
    logger.error(f"Database setup error: {e}")

# =====
# 3. PYDANTIC SCHEMAS & VALIDATION
# =====

class UserRegisterSchema(BaseModel):
    username: str = Field(..., min_length=3, max_length=20)
    email: str
    password: str

    # -----
    # Username Validation
    # -----

    @field_validator("username")
    @classmethod
    def username_alphanumeric(cls, v):
        if not re.match(r"^[a-zA-Z0-9_]+$", v):
            raise ValueError(
                "Username must contain only letters, numbers, and underscores."
            )

        return v

    # -----
    # Email Validation
    # -----

    @field_validator("email")
    @classmethod
    def validate_email(cls, v):
        v = v.strip().lower()

        if not re.match(
            r"^[a-zA-Z0-9._%+-]+@(gmail\.com|yahoo\.com)$",
            v
        ):
            raise ValueError(
                "Email must be a valid @gmail.com or @yahoo.com address."
            )

        return v

```

```

# -----
# Password Validation
# -----

@field_validator("password")
@classmethod
def validate_password(cls, v):

    if len(v) < 8:
        raise ValueError(
            "Password must contain at least 8 characters."
        )

    if not re.search(r"[A-Z]", v):
        raise ValueError(
            "Password must contain at least one uppercase letter."
        )

    if not re.search(r"[0-9]", v):
        raise ValueError(
            "Password must contain at least one number."
        )

    if not re.search(r"[@#$%^&*]", v):
        raise ValueError(
            "Password must contain at least one special character (@#$%^&*)."
        )

    return v

class HardwareSchema(BaseModel):
    item_name: str = Field(..., min_length=2, max_length=100)
    category: str = Field(..., min_length=2, max_length=50)
    quantity: int = Field(..., ge=0)
    unit_price: float = Field(..., ge=0.0)

def calculate_status(quantity: int) -> str:

    if quantity > 5:
        return "In Stock"

    elif 1 <= quantity <= 5:
        return "Low Stock"

    else:
        return "Out of Stock"

# =====

```

```

# 4. PASSWORD REQUIREMENT CHECKER
# =====

def get_password_requirements(password):
    """
    Returns a list containing the password requirements
    that have not yet been satisfied.
    """

    missing = []

    if len(password) < 8:
        missing.append("at least 8 characters")

    if not re.search(r"[A-Z]", password):
        missing.append("one uppercase letter")

    if not re.search(r"[0-9]", password):
        missing.append("one number")

    if not re.search(r"[@#$%^&]", password):
        missing.append("one special character (@#$%^&)")

    return missing

# =====
# 5. CONTROLLER / BACKEND LOGIC
# =====

class InventoryController:

    LOCKOUT_DURATION = 30
    MAX_FAILED_ATTEMPTS = 3

    def __init__(self, db_name=DB_NAME):
        self.db_name = db_name

    # =====
    # USER REGISTRATION
    # =====

    def register_user(self, username, email, password):

        try:
            validated = UserRegisterSchema(
                username=username,
                email=email,
                password=password
            )

```

```

except ValidationError as e:

    err_msg = e.errors()[0]["msg"]

    logger.warning(
        f"Registration validation failure: {err_msg}"
    )

    return False, f"Validation Error: {err_msg}"

conn = None

try:
    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()

    cursor.execute(
        "SELECT username, email FROM users WHERE LOWER(username) = LOWER(?) "
        "OR LOWER(email) = LOWER(?)",
        (validated.username, validated.email)
    )

    existing_user = cursor.fetchone()

    if existing_user:
        logger.warning(
            f"Registration attempt failed for '{validated.username}' / "
            f"'{validated.email}' because the account already exists."
        )
        return False, "Username or email already registered. Please use "
        "different details or log in."

    hashed_pw = bcrypt.hashpw(
        validated.password.encode("utf-8"),
        bcrypt.gensalt()
    )

    cursor.execute("""
        INSERT INTO users
        (
            username,
            email,
            password_hash,
            failed_attempts,
            locked_until
        )
        VALUES (?, ?, ?, 0, 0)
    """, (
        validated.username,
        validated.email,
        hashed_pw.decode("utf-8")
    ))

```

```

    ))

    conn.commit()

    logger.info(
        f"Account created: '{validated.username}'"
    )

    return True, "Registration successful! You may now log in."

except sqlite3.IntegrityError:

    return False, "Username or email already registered. Please use
different details."

except sqlite3.Error as e:

    logger.error(
        f"Registration database error: {e}"
    )

    return False, "Database error occurred during registration. Please try
again."

except Exception as e:

    logger.error(
        f"Unexpected registration error: {e}"
    )

    return False, "An unexpected error occurred during registration. Please
try again."

finally:

    if conn is not None:
        conn.close()

# =====
# LOGIN
# =====

def login_user(self, username, password):

    if not username or not password:

        return False, "Please enter both username and password."

    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()

```

```

cursor.execute("""
    SELECT
        password_hash,
        failed_attempts,
        locked_until
    FROM users
    WHERE username = ?
""", (username,))

row = cursor.fetchone()

# -----
# User Does Not Exist
# -----

if not row:

    conn.close()

    logger.warning(
        f"Failed login attempt for unknown user '{username}'"
    )

    return False, "Invalid username or password."

password_hash = row[0]
failed_attempts = row[1] or 0
locked_until = row[2] or 0

current_time = time.time()

# =====
# CHECK TEMPORARY LOCKOUT
# =====

if locked_until > current_time:

    remaining = int(locked_until - current_time) + 1

    conn.close()

    logger.warning(
        f"Login blocked for locked account '{username}'. "
        f"Remaining: {remaining} seconds."
    )

    return False, (
        f"Account temporarily locked. "
        f>Please try again in {remaining} seconds."
    )

```

```

# =====
# LOCKOUT EXPIRED
# =====

if locked_until != 0 and locked_until <= current_time:

    failed_attempts = 0

    cursor.execute("""
        UPDATE users
        SET failed_attempts = 0,
            locked_until = 0
        WHERE username = ?
    """, (username,))

    conn.commit()

# =====
# CHECK PASSWORD
# =====

password_correct = bcrypt.checkpw(
    password.encode("utf-8"),
    password_hash.encode("utf-8")
)

# =====
# SUCCESSFUL LOGIN
# =====

if password_correct:

    cursor.execute("""
        UPDATE users
        SET failed_attempts = 0,
            locked_until = 0
        WHERE username = ?
    """, (username,))

    conn.commit()
    conn.close()

    logger.info(
        f"User '{username}' logged in successfully."
    )

    return True, "Login successful!"

# =====
# FAILED LOGIN
# =====

```

```

failed_attempts += 1

if failed_attempts >= self.MAX_FAILED_ATTEMPTS:

    lockout_time = time.time() + self.LOCKOUT_DURATION

    cursor.execute("""
        UPDATE users
        SET failed_attempts = ?,
            locked_until = ?
        WHERE username = ?
    """, (
        failed_attempts,
        lockout_time,
        username
    ))

    conn.commit()
    conn.close()

    logger.warning(
        f"Account '{username}' locked for "
        f"{self.LOCKOUT_DURATION} seconds."
    )

    return False, (
        "Too many failed login attempts.\n"
        "Your account has been temporarily locked "
        "for 30 seconds."
    )

else:

    cursor.execute("""
        UPDATE users
        SET failed_attempts = ?
        WHERE username = ?
    """, (
        failed_attempts,
        username
    ))

    conn.commit()
    conn.close()

    remaining_attempts = (
        self.MAX_FAILED_ATTEMPTS - failed_attempts
    )

    logger.warning(

```

```

        f"Failed login attempt for '{username}'. "
        f"Attempts: {failed_attempts}"
    )

    return False, (
        "Invalid username or password.\n"
        f"Attempts remaining before lockout: "
        f"{remaining_attempts}"
    )

# =====
# FETCH HARDWARE
# =====

def fetch_allHardware(self):

    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()

    cursor.execute("""
        SELECT
            item_id,
            item_name,
            category,
            quantity,
            unit_price,
            status
        FROM hardware
    """)

    rows = cursor.fetchall()

    conn.close()

    return rows

# =====
# TOTAL ASSET VALUE
# =====

def get_total_asset_value(self):

    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()

    cursor.execute("""
        SELECT SUM(quantity * unit_price)
        FROM hardware
    """)

    total = cursor.fetchone()[0]

```

```

        conn.close()

        return total if total is not None else 0.0

# =====
# ADD HARDWARE
# =====

def addHardware(
    self,
    item_name,
    category,
    quantity_str,
    unit_price_str
):

    try:

        qty = int(quantity_str)
        price = float(unit_price_str)

        validated = HardwareSchema(
            item_name=item_name,
            category=category,
            quantity=qty,
            unit_price=price
        )

    except (ValueError, ValidationError) as e:

        if isinstance(e, ValidationError):
            err_msg = e.errors()[0]["msg"]
        else:
            err_msg = "Invalid numbers."

        logger.warning(
            f"Hardware validation failed: {err_msg}"
        )

        return False, f"Validation Error: {err_msg}"

    status = calculate_status(validated.quantity)

    conn = None

    try:

        conn = sqlite3.connect(self.db_name)
        cursor = conn.cursor()

```

```

# Case-insensitive duplicate check

cursor.execute("""
    SELECT item_id
    FROM hardware
    WHERE LOWER(item_name) = LOWER(?)
""", (
    validated.item_name.strip(),
))

if cursor.fetchone():

    logger.warning(
        f"Duplicate entry attempt: "
        f"'{validated.item_name}'"
    )

    return False, (
        f"An item named '{validated.item_name}' "
        f"already exists in inventory."
    )

cursor.execute("""
    INSERT INTO hardware
    (
        item_name,
        category,
        quantity,
        unit_price,
        status
    )
    VALUES (?, ?, ?, ?, ?)
""", (
    validated.item_name.strip(),
    validated.category.strip(),
    validated.quantity,
    validated.unit_price,
    status
))

conn.commit()

logger.info(
    f"New equipment added: '{validated.item_name}'"
)

return True, "Equipment added successfully!"

except sqlite3.IntegrityError:

    logger.warning(

```

```

        f"Integrity violation adding hardware: "
        f"'{validated.item_name}'"
    )

    return False, (
        f"An item named '{validated.item_name}' "
        f"already exists!"
    )

except sqlite3.Error as e:

    logger.error(
        f"Database error on adding hardware: {e}"
    )

    return False, f"Database Error: {e}"

finally:

    if conn:
        conn.close()

# =====
# UPDATE HARDWARE
# =====

def update_hardware(
    self,
    item_id,
    quantity_str,
    unit_price_str
):

    conn = None

    try:

        conn = sqlite3.connect(self.db_name)
        cursor = conn.cursor()

        cursor.execute("""
            SELECT quantity, unit_price
            FROM hardware
            WHERE item_id = ?
            """, (item_id,))

        current_record = cursor.fetchone()

        if not current_record:

            return False, (

```

```

        "Selected record could not be found."
    )

    current_qty, current_price = current_record

    new_qty = (
        int(quantity_str)
        if quantity_str.strip()
        else current_qty
    )

    new_price = (
        float(unit_price_str)
        if unit_price_str.strip()
        else current_price
    )

    if new_qty < 0 or new_price < 0:

        raise ValueError(
            "Quantity and price must be non-negative."
        )

    new_status = calculate_status(new_qty)

    cursor.execute("""
        UPDATE hardware
        SET
            quantity = ?,
            unit_price = ?,
            status = ?
        WHERE item_id = ?
    """, (
        new_qty,
        new_price,
        new_status,
        item_id
    ))

    conn.commit()

    logger.info(
        f"Hardware ID {item_id} updated. "
        f"New Qty: {new_qty}, "
        f"New Price: {new_price}"
    )

    return True, "Record updated successfully!"

except ValueError:

```

```

        logger.warning(
            f"Update failed for item ID {item_id}: "
            f"Invalid numeric input."
        )

        return False, (
            "Quantity must be a valid non-negative "
            "integer and price a valid number."
        )

    except sqlite3.Error as e:

        logger.error(
            f"Database update error for ID {item_id}: {e}"
        )

        return False, "Database update failed."

    finally:

        if conn:
            conn.close()

# =====
# CSV EXPORT
# =====

def export_to_csv(
    self,
    filename="inventory_report.csv"
):

    try:

        rows = self.fetch_all_hardware()

        timestamp = datetime.now().strftime(
            "%Y-%m-%d %H:%M:%S"
        )

        total_valuation = 0.0
        formatted_rows = []

        for r in rows:

            (
                item_id,
                item_name,
                category,
                qty,
                unit_price,
            )

```

```

        status
    ) = r

    item_total = qty * unit_price

    total_valuation += item_total

    formatted_rows.append([
        item_id,
        item_name,
        category,
        qty,
        f"${unit_price:,.2f}",
        f"${item_total:,.2f}",
        status
    ])

with open(
    filename,
    mode="w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.writer(file)

    writer.writerow([
        "=====
    ])

    writer.writerow([
        "CAMPUS HARDWARE INVENTORY MANAGEMENT REPORT"
    ])

    writer.writerow([
        f"Generated On: {timestamp}"
    ])

    writer.writerow([
        f"Total Recorded Items: {len(rows)}"
    ])

    writer.writerow([
        "=====
    ])

    writer.writerow([
        "Item ID",
        "Item Name",

```

```

        "Category",
        "Quantity",
        "Unit Price",
        "Total Price",
        "Status"
    ])

    writer.writerow(formatted_rows)

    writer.writerow([])

    writer.writerow([
        "",
        "",
        "",
        "",
        "",
        "GRAND TOTAL ASSET VALUE:",
        f"${total_valuation:,.2f}",
        ""
    ])

    logger.info(
        f"CSV Export generated: {filename}. "
        f"Total Value: ${total_valuation:,.2f}"
    )

    return True, (
        f"Report successfully exported to "
        f"{filename}!"
    )

except Exception as e:

    logger.error(
        f"CSV Export failed: {e}"
    )

    return False, f"Export failed: {e}"

# =====
# 6. LOGIN / REGISTER WINDOW
# =====

class LoginWindow:

    def __init__(self, root, on_success):

        self.root = root
        self.on_success = on_success
        self.controller = InventoryController()

```

```
self.root.title("Campus Hardware Inventory - Authentication")
self.root.geometry("500x520")
self.root.resizable(False, False)
```

```
# =====
# TITLE
# =====
```

```
tk.Label(
    root,
    text="Campus Hardware Inventory",
    font=("Arial", 18, "bold")
).pack(pady=(20, 5))
```

```
tk.Label(
    root,
    text="User Authentication",
    font=("Arial", 11)
).pack(pady=(0, 15))
```

```
# =====
# NOTEBOOK / SEPARATE PANES
# =====
```

```
self.notebook = ttk.Notebook(root)
```

```
self.login_frame = tk.Frame(self.notebook)
self.register_frame = tk.Frame(self.notebook)
```

```
self.notebook.add(
    self.login_frame,
    text=" Login "
)
```

```
self.notebook.add(
    self.register_frame,
    text=" Register "
)
```

```
self.notebook.pack(
    fill="both",
    expand=True,
    padx=25,
    pady=10
)
```

```
self.create_login_pane()
self.create_register_pane()
```

```
# =====
```

```

# LOGIN PANE
# =====

def create_login_pane(self):

    frame = self.login_frame

    # Username

    tk.Label(
        frame,
        text="Username:"
    ).pack(anchor="w", padx=40, pady=(30, 5))

    self.login_username = tk.Entry(
        frame,
        width=38
    )

    self.login_username.pack(
        padx=40,
        pady=(0, 15)
    )

    # Password

    tk.Label(
        frame,
        text="Password:"
    ).pack(anchor="w", padx=40, pady=(0, 5))

    self.login_password = tk.Entry(
        frame,
        show="*",
        width=38
    )

    self.login_password.pack(
        padx=40
    )

    # =====
    # SHOW / HIDE PASSWORD
    # =====

    self.login_show_password = tk.BooleanVar(
        value=False
    )

    tk.Checkbutton(
        frame,

```

```

        text="Show Password",
        variable=self.login_show_password,
        command=self.toggle_login_password
    ).pack(
        anchor="w",
        padx=40,
        pady=(0, 15)
    )

    # =====
    # LOGIN BUTTON
    # =====

    tk.Button(
        frame,
        text="Login",
        command=self.handle_login,
        bg="#4CAF50",
        fg="white",
        width=18,
        height=2
    ).pack(pady=10)

    # =====
    # REGISTER PANE
    # =====

def create_register_pane(self):

    frame = self.register_frame

    # Username

    tk.Label(
        frame,
        text="Username:"
    ).pack(
        anchor="w",
        padx=40,
        pady=(20, 5)
    )

    self.register_username = tk.Entry(
        frame,
        width=38
    )

    self.register_username.pack(
        padx=40,
        pady=(0, 10)
    )

```

```
)

# Email

tk.Label(
    frame,
    text="Email Address:"
).pack(
    anchor="w",
    padx=40,
    pady=(0, 5)
)

self.register_email = tk.Entry(
    frame,
    width=38
)

self.register_email.pack(
    padx=40,
    pady=(0, 3)
)

tk.Label(
    frame,
    text="Only @gmail.com or @yahoo.com addresses are accepted.",
    fg="gray",
    font=("Arial", 8)
).pack(
    anchor="w",
    padx=40,
    pady=(0, 10)
)

# Password

tk.Label(
    frame,
    text="Password:"
).pack(
    anchor="w",
    padx=40,
    pady=(0, 5)
)

self.register_password = tk.Entry(
    frame,
    show="*",
    width=38
)
```

```

self.register_password.pack(
    padx=40
)

# =====
# PASSWORD REQUIREMENTS NOTE
# =====

self.register_password_note = tk.Label(
    frame,
    text=(
        "Password requirements:\n"
        "• At least 8 characters\n"
        "• One uppercase letter\n"
        "• One number\n"
        "• One special character (@#$%^&*)"
    ),
    fg="red",
    font=("Arial", 9),
    wraplength=360,
    justify="left"
)

self.register_password_note.pack(
    anchor="w",
    padx=40,
    pady=(5, 5)
)

self.register_password.bind(
    "<KeyRelease>",
    self.update_register_password_note
)

# =====
# SHOW / HIDE PASSWORD
# =====

self.register_show_password = tk.BooleanVar(
    value=False
)

tk.Checkbutton(
    frame,
    text="Show Password",
    variable=self.register_show_password,
    command=self.toggle_register_password
).pack(
    anchor="w",
    padx=40,
    pady=(0, 10)
)

```

```

    )

    # =====
    # REGISTER BUTTON
    # =====

    tk.Button(
        frame,
        text="Create Account",
        command=self.handle_register,
        bg="#2196F3",
        fg="white",
        width=18,
        height=2
    ).pack(pady=10)

    # =====
    # PASSWORD NOTE - REGISTER
    # =====

    def update_register_password_note(self, event=None):

        password = self.register_password.get()

        missing = get_password_requirements(password)

        if not password:

            self.register_password_note.config(
                text=(
                    "Password requirements:\n"
                    "• At least 8 characters\n"
                    "• One uppercase letter\n"
                    "• One number\n"
                    "• One special character (@#$$%^&*)"
                ),
                fg="red"
            )

        elif missing:

            self.register_password_note.config(
                text="Password is missing:\n• "
                + "\n• ".join(missing),
                fg="red"
            )

        else:

            self.register_password_note.config(
                text="✓ Password meets all requirements.",

```

```

        fg="green"
    )

# =====
# SHOW / HIDE LOGIN PASSWORD
# =====

def toggle_login_password(self):

    if self.login_show_password.get():

        self.login_password.config(
            show=""
        )

    else:

        self.login_password.config(
            show="*"
        )

# =====
# SHOW / HIDE REGISTER PASSWORD
# =====

def toggle_register_password(self):

    if self.register_show_password.get():

        self.register_password.config(
            show=""
        )

    else:

        self.register_password.config(
            show="*"
        )

# =====
# LOGIN
# =====

def handle_login(self):

    user = self.login_username.get().strip()
    pwd = self.login_password.get()

    success, msg = self.controller.login_user(
        user,
        pwd
    )

```

```

    )

    if success:

        messagebox.showinfo(
            "Success",
            msg
        )

        self.on_success(user)

    else:

        messagebox.showerror(
            "Login Failed",
            msg
        )

# =====
# REGISTER
# =====

def handle_register(self):

    user = self.register_username.get().strip()
    email = self.register_email.get().strip()
    pwd = self.register_password.get()

    success, msg = self.controller.register_user(
        user,
        email,
        pwd
    )

    if success:

        messagebox.showinfo(
            "Registration Successful",
            msg
        )

        # Clear registration form

        self.register_username.delete(
            0,
            tk.END
        )

        self.register_email.delete(
            0,
            tk.END

```

```

    )

    self.register_password.delete(
        0,
        tk.END
    )

    self.update_register_password_note()

    # Switch back to login tab

    self.notebook.select(
        self.login_frame
    )

    # Pre-fill username

    self.login_username.delete(
        0,
        tk.END
    )

    self.login_username.insert(
        0,
        user
    )

else:

    messagebox.showwarning(
        "Registration Alert",
        msg
    )

# =====
# 7. INVENTORY WINDOW
# =====

class InventoryWindow:

    def __init__(
        self,
        root,
        username,
        on_logout
    ):

        self.root = root
        self.username = username
        self.on_logout = on_logout

```

```

self.controller = InventoryController()

self.root.title(
    f"Campus Hardware Inventory - User: {self.username}"
)

self.root.geometry(
    "920x750"
)

# =====
# HEADER VALUATION
# =====

self.lbl_valuation = tk.Label(
    self.root,
    text="Total Asset Value: $0.00",
    font=("Arial", 12, "bold"),
    bg="#333",
    fg="white",
    pady=10
)

self.lbl_valuation.pack(
    fill="x"
)

# =====
# ADD EQUIPMENT FORM
# =====

frame_form = tk.LabelFrame(
    self.root,
    text="Add New Equipment",
    padx=10,
    pady=10
)

frame_form.pack(
    fill="x",
    padx=10,
    pady=5
)

inner_form = tk.Frame(frame_form)

inner_form.pack(
    side="left",
    padx=5
)

```

```
# Name

tk.Label(
    inner_form,
    text="Name:"
).grid(
    row=0,
    column=0,
    padx=5,
    pady=5,
    sticky="e"
)

self.ent_name = tk.Entry(
    inner_form,
    width=20
)

self.ent_name.grid(
    row=0,
    column=1,
    padx=5,
    pady=5
)

# Category

tk.Label(
    inner_form,
    text="Category:"
).grid(
    row=0,
    column=2,
    padx=5,
    pady=5,
    sticky="e"
)

self.ent_cat = tk.Entry(
    inner_form,
    width=20
)

self.ent_cat.grid(
    row=0,
    column=3,
    padx=5,
    pady=5
)

# Quantity
```

```
tk.Label(  
    inner_form,  
    text="Quantity:"  
) .grid(  
    row=1,  
    column=0,  
    padx=5,  
    pady=5,  
    sticky="e"  
)  
  
self.ent_qty = tk.Entry(  
    inner_form,  
    width=20  
)  
  
self.ent_qty.grid(  
    row=1,  
    column=1,  
    padx=5,  
    pady=5  
)  
  
# Price  
  
tk.Label(  
    inner_form,  
    text="Unit Price ($):"  
) .grid(  
    row=1,  
    column=2,  
    padx=5,  
    pady=5,  
    sticky="e"  
)  
  
self.ent_price = tk.Entry(  
    inner_form,  
    width=20  
)  
  
self.ent_price.grid(  
    row=1,  
    column=3,  
    padx=5,  
    pady=5  
)  
  
# Save
```

```

tk.Button(
    inner_form,
    text="Save Item",
    command=self.add_item,
    bg="#4CAF50",
    fg="white",
    padx=10
).grid(
    row=0,
    column=4,
    rowspan=2,
    padx=15,
    pady=5,
    sticky="nsew"
)

# =====
# STATUS LEGEND
# =====

frame_legend = tk.LabelFrame(
    frame_form,
    text="Status Legend",
    padx=10,
    pady=5
)

frame_legend.pack(
    side="right",
    padx=15,
    pady=5
)

tk.Label(
    frame_legend,
    text="In Stock (> 5)",
    bg="#d4edda",
    fg="#155724",
    font=("Arial", 8, "bold"),
    relief="solid",
    bd=1,
    anchor="center"
).grid(
    row=0,
    column=0,
    padx=5,
    pady=2,
    sticky="ew"
)

tk.Label(

```

```

        frame_legend,
        text="Low Stock (1-5)",
        bg="#fff3cd",
        fg="#856404",
        font=("Arial", 8, "bold"),
        relief="solid",
        bd=1,
        anchor="center"
    ).grid(
        row=1,
        column=0,
        padx=5,
        pady=2,
        sticky="ew"
    )

    tk.Label(
        frame_legend,
        text="Out of Stock (0)",
        bg="#f8d7da",
        fg="#721c24",
        font=("Arial", 8, "bold"),
        relief="solid",
        bd=1,
        anchor="center"
    ).grid(
        row=2,
        column=0,
        padx=5,
        pady=2,
        sticky="ew"
    )

# =====
# DATA TABLE
# =====

frame_table = tk.Frame(
    self.root
)

frame_table.pack(
    fill="both",
    expand=True,
    padx=10,
    pady=5
)

scroll_y = tk.Scrollbar(
    frame_table,
    orient=tk.VERTICAL

```

```

    )

    self.tree = ttk.Treeview(
        frame_table,
        columns=(
            "ID",
            "Name",
            "Category",
            "Qty",
            "Price",
            "Status"
        ),
        show="headings",
        yscrollcommand=scroll_y.set
    )

    scroll_y.config(
        command=self.tree.yview
    )

    scroll_y.pack(
        side=tk.RIGHT,
        fill=tk.Y
    )

    for col in self.tree["columns"]:

        self.tree.heading(
            col,
            text=col
        )

        self.tree.column(
            col,
            width=120,
            anchor="center"
        )

    self.tree.tag_configure(
        "In Stock",
        background="#d4edda"
    )

    self.tree.tag_configure(
        "Low Stock",
        background="#fff3cd"
    )

    self.tree.tag_configure(
        "Out of Stock",
        background="#f8d7da"
    )

```

```

    )

    self.tree.pack(
        fill="both",
        expand=True
    )

    # =====
    # UPDATE FRAME
    # =====

    frame_update = tk.LabelFrame(
        self.root,
        text="Update Selected Record",
        padx=10,
        pady=5
    )

    frame_update.pack(
        fill="x",
        padx=10,
        pady=5
    )

    inner_update = tk.Frame(
        frame_update
    )

    inner_update.pack()

    tk.Label(
        inner_update,
        text="New Quantity:"
    ).grid(
        row=0,
        column=0,
        padx=5,
        pady=5,
        sticky="e"
    )

    self.ent_up_qty = tk.Entry(
        inner_update,
        width=18
    )

    self.ent_up_qty.grid(
        row=0,
        column=1,
        padx=5,
        pady=5
    )

```

```

    )

    tk.Label(
        inner_update,
        text="New Unit Price ($):"
    ).grid(
        row=0,
        column=2,
        padx=5,
        pady=5,
        sticky="e"
    )

    self.ent_up_price = tk.Entry(
        inner_update,
        width=18
    )

    self.ent_up_price.grid(
        row=0,
        column=3,
        padx=5,
        pady=5
    )

    tk.Button(
        inner_update,
        text="Update Record",
        command=self.update_item,
        bg="#2196F3",
        fg="white",
        padx=10
    ).grid(
        row=0,
        column=4,
        padx=15,
        pady=5,
        sticky="ew"
    )

    # =====
    # BOTTOM CONTROLS
    # =====

    frame_controls = tk.Frame(
        self.root
    )

    frame_controls.pack(
        fill="x",
        padx=10,

```

```

        pady=10
    )

    tk.Button(
        frame_controls,
        text="Export Inventory to CSV",
        command=self.export_csv,
        bg="#17a2b8",
        fg="white"
    ).pack(
        side="left",
        padx=5
    )

    tk.Button(
        frame_controls,
        text="Logout",
        command=self.logout,
        bg="#dc3545",
        fg="white"
    ).pack(
        side="right",
        padx=5
    )

    self.refresh_data()

# =====
# REFRESH TABLE
# =====

def refresh_data(self):

    for row in self.tree.get_children():

        self.tree.delete(row)

    rows = self.controller.fetch_all_hardware()

    for r in rows:

        status = r[5]

        self.tree.insert(
            "",
            tk.END,
            values=r,
            tags=(status,)
        )

    total_val = (

```

```

        self.controller.get_total_asset_value()
    )

    self.lbl_valuation.config(
        text=f"Total Asset Value: ${total_val:,.2f}"
    )

# =====
# ADD ITEM
# =====

def add_item(self):

    name = self.ent_name.get().strip()
    cat = self.ent_cat.get().strip()
    qty = self.ent_qty.get().strip()
    price = self.ent_price.get().strip()

    success, msg = (
        self.controller.add_hardware(
            name,
            cat,
            qty,
            price
        )
    )

    if success:

        messagebox.showinfo(
            "Success",
            msg
        )

        self.ent_name.delete(
            0,
            tk.END
        )

        self.ent_cat.delete(
            0,
            tk.END
        )

        self.ent_qty.delete(
            0,
            tk.END
        )

        self.ent_price.delete(
            0,

```

```

        tk.END
    )

    self.refresh_data()

else:

    messagebox.showwarning(
        "Duplicate or Invalid Entry",
        msg
    )

# =====
# UPDATE ITEM
# =====

def update_item(self):

    selected_item = self.tree.selection()

    if not selected_item:

        messagebox.showwarning(
            "Selection Error",
            "Please select a row from the table to update!"
        )

        return

    item_values = self.tree.item(
        selected_item,
        "values"
    )

    item_id = item_values[0]

    qty_str = self.ent_up_qty.get().strip()
    price_str = self.ent_up_price.get().strip()

    success, msg = (
        self.controller.update_hardware(
            item_id,
            qty_str,
            price_str
        )
    )

    if success:

        messagebox.showinfo(
            "Success",

```

```

        msg
    )

    self.ent_up_qty.delete(
        0,
        tk.END
    )

    self.ent_up_price.delete(
        0,
        tk.END
    )

    self.refresh_data()

else:

    messagebox.showerror(
        "Error",
        msg
    )

# =====
# EXPORT CSV
# =====

def export_csv(self):

    success, msg = (
        self.controller.export_to_csv()
    )

    if success:

        messagebox.showinfo(
            "Export Successful",
            msg
        )

    else:

        messagebox.showerror(
            "Export Failed",
            msg
        )

# =====
# LOGOUT
# =====

def logout(self):

```

```

        logger.info(
            f"User '{self.username}' logged out."
        )

        self.on_logout()

# =====
# 8. APP LAUNCHER / WINDOW MANAGER
# =====

def launch_main_app(username):

    for widget in root.winfo_children():

        widget.destroy()

    InventoryWindow(
        root,
        username,
        on_logout=launch_login
    )

def launch_login():

    for widget in root.winfo_children():

        widget.destroy()

    LoginWindow(
        root,
        on_success=launch_main_app
    )

# =====
# 9. PROGRAM ENTRY POINT
# =====

if __name__ == "__main__":

    init_db()

    root = tk.Tk()

    launch_login()

    root.mainloop()

```